

Базы данных
Лабораторная работа 6

Романенко Мария Б23-513

Триггеры и хранимые процедуры

Цель: Реализовать бизнес-логику на уровне базы данных.

Триггеры и процедуры должны:

- реализовывать реальные бизнес-правила;
- обрабатывать негативные сценарии;
- корректно работать при множественных вставках.

1. Триггер: Запрет повторной продажи/аренды недвижимости

- **Суть правила:** Объект недвижимости (Property) не может быть продан повторно, если по нему уже есть активный договор SaleContract. Также объект не может быть сдан в аренду RentContract, если текущая дата попадает в диапазон дат уже существующего договора аренды (проверка пересечения сроков).
- **Негативный сценарий:** Попытка зарегистрировать сделку по занятому объекту вызывает ошибку EXCEPTION.

```
CREATE OR REPLACE FUNCTION check_double_sale()
RETURNS TRIGGER AS $$
BEGIN
    -- Ищем, есть ли уже договор купли-продажи на этот объект
    -- (исключаем текущий ID, чтобы не блокировать UPDATE самого себя)
    IF EXISTS (
        SELECT 1 FROM SaleContract
        WHERE PropertyId = NEW.PropertyId
        AND ContractId != COALESCE(NEW.ContractId, -1)
    ) THEN
        RAISE EXCEPTION 'Ошибка: Объект недвижимости с ID % уже
        продан!', NEW.PropertyId;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER trg_prevent_double_sale
BEFORE INSERT OR UPDATE ON SaleContract
FOR EACH ROW
EXECUTE PROCEDURE check_double_sale();
```

Query

Query History

1

SELECT * FROM Property;

2

3

DELETE FROM SaleContract WHERE PropertyId IN (SELECT PropertyId FROM Property WHERE Address = 'тест триггера');

4

DELETE FROM Property WHERE Address = 'тест триггера';

5

6

SELECT * FROM Property;

7

8

INSERT INTO Property (Type, Area, Address, RegionId, ManagerId)

9

VALUES ('квартира', 55.5, 'тест триггера',

10

(SELECT MAX(RegionId) FROM Region),

11

(SELECT MAX(EmployeeId) FROM Manager));

12

13

SELECT * FROM Property

14

WHERE Address = 'тест триггера'

15

ORDER BY PropertyId DESC;

16

Data Output

Messages

Notifications

+

📄

📄

📄

🗑️

📄

📄

📄

SQL

Showing rows: 1 to 1

	propertyid [PK] integer	type character varying (50)	area numeric (10,2)	address character varying (255)	floor integer	ceilingheight numeric (10,2)	regionid integer	managerid integer	
1	93	квартира	55.50	тест триггера	1	3.00	20	5	

Query

Query History

23

INSERT INTO SaleContract (PropertyId, BuyerId, SellerId, ManagerId, RealtorId, BankId, TotalPrice, AgencyFee, PaymentType, ContractDate

24

VALUES (

25

(SELECT MAX(PropertyId) FROM Property WHERE Address = 'тест триггера'),

26

(SELECT MAX(ClientId) FROM Client),

27

(SELECT MIN(ClientId) FROM Client),

28

(SELECT MAX(EmployeeId) FROM Manager),

29

(SELECT MAX(EmployeeId) FROM Realtor),

30

(SELECT MAX(BankId) FROM Bank),

31

10000000, 500000, 'наличные', CURRENT_DATE

32

);

33

34

SELECT sc.ContractId, sc.TotalPrice, sc.ContractDate, p.Address

35

FROM SaleContract sc

36

JOIN Property p ON sc.PropertyId = p.PropertyId

37

WHERE p.PropertyId = (SELECT MAX(PropertyId) FROM Property WHERE Address = 'тест триггера');

38

Data Output

Messages

Notifications

+

📄

📄

📄

🗑️

📄

📄

📄

SQL

Showing rows: 1 to 1

Page No: 1

	contractid integer	totalprice numeric (15,2)	contractdate date	address character varying (255)	
1	99	10000000.00	2026-05-20	тест триггера	

39

INSERT INTO SaleContract (PropertyId, BuyerId, SellerId, ManagerId, RealtorId, BankId, TotalPrice, AgencyFee, PaymentType, ContractDate)

40

VALUES (

41

(SELECT MAX(PropertyId) FROM Property WHERE Address = 'тест триггера'),

42

(SELECT MAX(ClientId) FROM Client),

43

(SELECT MIN(ClientId) FROM Client),

44

(SELECT MAX(EmployeeId) FROM Manager),

45

(SELECT MAX(EmployeeId) FROM Realtor),

46

(SELECT MAX(BankId) FROM Bank),

47

15000000, 750000, 'наличные', CURRENT_DATE

48

);

Data Output

Messages

Notifications

ERROR: Ошибка: Объект недвижимости с ID 93 уже продан!

CONTEXT: функция PL/pgSQL check_double_sale(), строка 10, оператор RAISE

ОШИБКА: Ошибка: Объект недвижимости с ID 93 уже продан!

SQL state: P0001

```

CREATE OR REPLACE FUNCTION check_rent_overlap()
RETURNS TRIGGER AS $$
BEGIN
    -- Проверяем, есть ли уже договор аренды на эту квартиру,
    -- сроки которого накладываются на сроки нового договора
    IF EXISTS (
        SELECT 1 FROM RentContract
        WHERE PropertyId = NEW.PropertyId
        AND ContractId != COALESCE(NEW.ContractId, -1)
        -- Сравниваем (Дата старта, Период) с (Датой старта, Периодом)
        существующих договоров
        AND (NEW.ContractDate, (NEW.DurationMonths ||
        ' months')::interval)
        OVERLAPS
        (ContractDate, (DurationMonths || ' months')::interval)
    ) THEN
        RAISE EXCEPTION 'Ошибка: Объект недвижимости с ID % уже сдан в
аренду на выбранные даты!', NEW.PropertyId;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

```

CREATE TRIGGER trg_prevent_double_rent
BEFORE INSERT OR UPDATE ON RentContract
FOR EACH ROW
EXECUTE PROCEDURE check_rent_overlap();

```


2. Триггер: Расчет вознаграждения агентства и риелтора

- **Суть правила:** При добавлении договора SaleContract или RentContract БД должна автоматически рассчитать AgencyFee. На основе Property -> Regio высчитывается финальная премия.
- **Негативный сценарий:** Если в договоре указана сумма, противоречащая минимальным правилам агентства, триггер корректирует её или блокирует операцию.

```
CREATE OR REPLACE FUNCTION calc_and_check_agency_fee()
RETURNS TRIGGER AS $$
DECLARE
    min_fee DECIMAL(15,2);
BEGIN
    min_fee := NEW.TotalPrice * 0.05;
    IF NEW.AgencyFee IS NULL OR NEW.AgencyFee < min_fee THEN
        -- RAISE NOTICE просто выводит предупреждение в консоль,
но не прерывает запрос
        RAISE NOTICE 'Указанная комиссия (%) слишком мала.
Установлен минимум: %', NEW.AgencyFee, min_fee;
        NEW.AgencyFee := min_fee;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_adjust_agency_fee
BEFORE INSERT OR UPDATE ON SaleContract
FOR EACH ROW
EXECUTE PROCEDURE calc_and_check_agency_fee();
```

Query

Query History

1

DELETE FROM SaleContract WHERE PropertyId IN (SELECT PropertyId FROM Property WHERE Address = 'тест триггера комиссии');

2

DELETE FROM Property WHERE Address = 'тест триггера комиссии';

3

4

5

INSERT INTO Property (Type, Area, Address, RegionId, ManagerId)

6

VALUES ('офис', 100.0, 'тест триггера комиссии',

7

(SELECT MAX(RegionId) FROM Region),

8

(SELECT MAX(EmployeeId) FROM Manager));

9

10

11

SELECT * FROM Property

12

WHERE Address = 'тест триггера комиссии'

13

ORDER BY PropertyId DESC;

14

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 1

	propertyid [PK] integer	type character varying (50)	area numeric (10,2)	address character varying (255)	floor integer	ceilingheight numeric (10,2)	regionid integer	managerid integer
1	95	офис	100.00	тест триггера комиссии	1	3.00	20	5

3. Процедура: Увольнение сотрудника

- **Суть правила:** Нельзя просто проставить FireDate сотруднику, если за ним закреплены активные объекты недвижимости Property.ManagerId или незавершенные договоры SaleContract/RentContract. Процедура принимает Id увольняемого сотрудника и Id преемника, переводит все дела на нового человека, проверяет отсутствие конфликтов и только потом проставляет дату увольнения.

```
CREATE OR REPLACE PROCEDURE fire_and_reassign_employee(
    p_fired_id INTEGER,
    p_successor_id INTEGER
)
LANGUAGE plpgsql AS $$
DECLARE
    v_is_successor_active BOOLEAN;
BEGIN
    -- Проверяем, существует ли преемник и работает ли он
    SELECT (FireDate IS NULL) INTO v_is_successor_active
    FROM Employee
    WHERE EmployeeId = p_successor_id;
    IF NOT FOUND OR v_is_successor_active = FALSE THEN
        RAISE EXCEPTION 'Негативный сценарий: Преемник (ID %) не найден или
уже уволен!', p_successor_id;
    END IF;
    -- передаем дела преемнику (обновляем связи)
    UPDATE Property SET ManagerId = p_successor_id WHERE ManagerId =
p_fired_id;
    UPDATE SaleContract SET RealtorId = p_successor_id WHERE RealtorId =
p_fired_id;
    UPDATE RentContract SET RealtorId = p_successor_id WHERE RealtorId =
p_fired_id;

    UPDATE Employee SET FireDate = CURRENT_DATE WHERE EmployeeId =
p_fired_id;
END;
$;
```

Query

Query History

1

DELETE FROM SaleContract WHERE RealtorId IN (SELECT EmployeeId FROM Employee WHERE FullName LIKE 'ТЕСТ %');

2

DELETE FROM RentContract WHERE RealtorId IN (SELECT EmployeeId FROM Employee WHERE FullName LIKE 'ТЕСТ %');

3

DELETE FROM Property WHERE ManagerId IN (SELECT EmployeeId FROM Employee WHERE FullName LIKE 'ТЕСТ %');

4

5

DELETE FROM Manager WHERE EmployeeId IN (SELECT EmployeeId FROM Employee WHERE FullName LIKE 'ТЕСТ %');

6

DELETE FROM Employee WHERE FullName LIKE 'ТЕСТ %';

7

8

9

INSERT INTO Employee (FullName, Phone, BirthDate, TaxId, PassportNumber, HireDate)

10

VALUES ('ТЕСТ Увольняемый', '+7(999)111-11-11', '1990-01-01', '111111111111', '1111 111111', '2020-01-01');

11

12

INSERT INTO Employee (FullName, Phone, BirthDate, TaxId, PassportNumber, HireDate)

13

VALUES ('ТЕСТ Преемник', '+7(999)222-22-22', '1990-01-01', '222222222222', '2222 222222', '2020-01-01');

14

15

INSERT INTO Manager (EmployeeId) SELECT EmployeeId FROM Employee WHERE FullName = 'ТЕСТ Увольняемый';

16

INSERT INTO Manager (EmployeeId) SELECT EmployeeId FROM Employee WHERE FullName = 'ТЕСТ Преемник';

17

18

19

INSERT INTO Property (Type, Area, Address, RegionId, ManagerId)

20

VALUES ('дом', 150.0, 'тест увольнения менеджера',

21

(SELECT MAX(RegionId) FROM Region),

22

(SELECT EmployeeId FROM Employee WHERE FullName = 'ТЕСТ Увольняемый'));

23

24

25

SELECT EmployeeId, FullName, HireDate, FireDate

26

FROM Employee

27

WHERE FullName LIKE 'ТЕСТ %';

Data Output

Messages

Notifications

+

SQL

Showing rows: 1 to 2

	employeeid [PK] integer	fullname character varying (255)	hiredate date	firedate date
1	49	ТЕСТ Увольняемый	2020-01-...	[null]
2	50	ТЕСТ Преемник	2020-01-...	[null]

30

SELECT PropertyId, Address, ManagerId

31

FROM Property

32

WHERE Address = 'тест увольнения менеджера';

33

34

Data Output

Messages

Notifications

+

SQL

	propertyid [PK] integer	address character varying (255)	managerid integer
1	100	тест увольнения менедже...	49

35

CALL fire_and_reassign_employee(49, 50);

36

37

38

SELECT EmployeeId, FullName, HireDate, FireDate

39

FROM Employee

40

WHERE FullName LIKE 'ТЕСТ %';

41

Data Output

Messages

Notifications

+

SQL

	employeeid [PK] integer	fullname character varying (255)	hiredate date	firedate date
1	50	ТЕСТ Преемник	2020-01-...	[null]
2	49	ТЕСТ Увольняемый	2020-01-...	2026-05-...

43

SELECT PropertyId, Address, ManagerId

44

FROM Property

45

WHERE Address = 'тест увольнения менеджера';

Data Output

Messages

Notifications

+

SQL

	propertyid [PK] integer	address character varying (255)	managerid integer
1	100	тест увольнения менедже...	50

4. Процедура: Начисление и массовое формирование исходящих платежей риелторам за закрытый месяц

- **Суть правила:** Процедура запускается, анализирует все закрытые договоры за последний месяц, рассчитывает чистые выплаты для каждого риелтора (с учетом его базовой ставки BasePercent и региональных бонусов) и выполняет массовую вставку со статусом 'не оплачен' и типом 'исходящий_риелтору'.
- **Негативный сценарий:** Запуск процедуры за уже рассчитанный и закрытый период блокируется во избежание дублирования выплат.

```
CREATE OR REPLACE PROCEDURE generate_monthly_payouts(p_target_month DATE)
LANGUAGE plpgsql AS $$
DECLARE
    v_start_date DATE := DATE_TRUNC('month', p_target_month);
    v_end_date DATE := (v_start_date + INTERVAL '1 month') - INTERVAL '1
day';
    v_already_exists BOOLEAN;
BEGIN
    -- проверяем, не создавали ли мы уже выплаты за этот месяц
    SELECT EXISTS (
        SELECT 1 FROM PaymentDocument pd
        JOIN SaleContract sc ON pd.ContractId = sc.ContractId
        WHERE pd.DocumentType = 'исходящий_риелтору'
        AND sc.ContractDate BETWEEN v_start_date AND v_end_date
    ) INTO v_already_exists;

    IF v_already_exists THEN
        RAISE EXCEPTION 'Выплаты за период % уже были сформированы! Двойное
начисление заблокировано.', v_start_date;
    END IF;

    -- Массовая вставка расчетов за месяц
    INSERT INTO PaymentDocument (
        ContractId, ContractType, DocumentType, Amount, CreationDate,
Status,
        RealtorId, ManagerId, AccountantId
    )
    SELECT
        sc.ContractId,
        'продажа',
        'исходящий_риелтору',
        sc.AgencyFee * (r.BasePercent / 100.0),
        CURRENT_TIMESTAMP,
        'не оплачен',
        sc.RealtorId,
        1,
```

```

1
FROM SaleContract sc
JOIN Realtor r ON sc.RealtorId = r.EmployeeId
WHERE sc.ContractDate BETWEEN v_start_date AND v_end_date;
END;
$$;

```

Query

Query History

```

1 DELETE FROM PaymentDocument WHERE ContractId IN (SELECT ContractId FROM SaleContract WHERE ContractDate = '2019-05-15');
2 DELETE FROM SaleContract WHERE ContractDate = '2019-05-15';
3 DELETE FROM Property WHERE Address = 'тест массовых выплат';
4
5 INSERT INTO Property (Type, Area, Address, RegionId, ManagerId)
6 VALUES ('квартира', 60.0, 'тест массовых выплат',
7         (SELECT MAX(RegionId) FROM Region),
8         (SELECT MAX(EmployeeId) FROM Manager));
9
10 INSERT INTO SaleContract (PropertyId, BuyerId, SellerId, ManagerId, RealtorId, BankId, TotalPrice, AgencyFee, PaymentType, ContractDate)
11 VALUES (
12         (SELECT MAX(PropertyId) FROM Property WHERE Address = 'тест массовых выплат'),
13         (SELECT MAX(ClientId) FROM Client),
14         (SELECT MIN(ClientId) FROM Client),
15         (SELECT MAX(EmployeeId) FROM Manager),
16         (SELECT MAX(EmployeeId) FROM Realtor),
17         (SELECT MAX(BankId) FROM Bank),
18         100000000,
19         500000,
20         'наличные',
21         '2019-05-15'
22 );
23
24 SELECT PaymentId, ContractId, Amount, DocumentType
25 FROM PaymentDocument
26 WHERE DocumentType = 'исходящий_риелтору'
27        AND ContractId = (SELECT MAX(ContractId) FROM SaleContract WHERE ContractDate = '2019-05-15');

```

Data Output

Messages

Notifications

paymentid

[PK] integer

contractid

integer

amount

numeric (15,2)

documenttype

character varying (50)

```

30 CALL generate_monthly_payouts('2019-05-01');
31
32
33 SELECT pd.PaymentId, pd.DocumentType, pd.Amount, pd.Status, pd.CreationDate, r.BasePercent
34 FROM PaymentDocument pd
35 JOIN SaleContract sc ON pd.ContractId = sc.ContractId
36 JOIN Realtor r ON sc.RealtorId = r.EmployeeId
37 WHERE sc.ContractDate = '2019-05-15';

```

Data Output

Messages

Notifications

	paymentid integer	documenttype character varying (50)	amount numeric (15,2)	status character varying (50)	creationdate timestamp without time zone	basepercent numeric (5,2)
1	62	исходящий_риелтору	15050.00	не оплачен	2026-05-20 03:34:30.51261	3.01

```

40 CALL generate_monthly_payouts('2019-05-01');
41
42 SELECT pd.PaymentId, pd.DocumentType, pd.Amount, pd.Status, pd.CreationDate, r.BasePercent
43 FROM PaymentDocument pd
44 JOIN SaleContract sc ON pd.ContractId = sc.ContractId
45 JOIN Realtor r ON sc.RealtorId = r.EmployeeId
46 WHERE sc.ContractDate = '2019-05-15';
47

```

Data Output

Messages

Notifications

ERROR: Выплаты за период 2019-05-01 уже были сформированы! Двойное начисление заблокировано.

CONTEXT: функция PL/pgSQL generate_monthly_payouts(date), строка 17, оператор RAISE

ОШИБКА: Выплаты за период 2019-05-01 уже были сформированы! Двойное начисление заблокировано.